

8K Labels — Design Document

Project: Wine Label Generator — Next.js migration **Repository:** `whiterabbitparks/wine-label-generator`

Status: Live in mock-generation mode · milestone `4da0504` **Last updated:** 2026-07-29

1. Overview

8K Labels is a wine-label generator for winemakers. The user enters their label details in an interactive layout preview, optionally describes a visual idea ("their story") and uploads a reference sketch, and receives the same label rendered in **six distinct design styles** as print-ready SVG — sized in real millimetres, with a 5 mm safety margin and 2 mm bleed suitable for production.

The product originally shipped as a **single self-contained 3.2 MB HTML file** (`8k-labels-package/dist/configurator.html`) with no server. This project wraps that application in a **Next.js host** so it gains the one thing a static file cannot have: a backend — initially for AI artwork generation, later for persistence, auth, and payments.

2. Goals and non-goals

Goals

1. **Pixel-perfect preservation.** The Next.js app must be visually and behaviorally indistinguishable from the original file. Every deviation is a defect by definition.
2. **A real backend.** Image generation through a server-side API route with the model API key never exposed to the browser.
3. **Provable correctness.** Automated parity gates, not human judgement, decide whether the port is faithful.

Non-goals (current phase)

- Mobile/responsive verification (explicitly deferred; parity is verified at 1440 px only).
- React-ification of the UI. The original vanilla-JS code runs as-is; rewriting it in React is possible later but is not a goal.
- Logo (box 2) placement rules; per-style image treatments — deferred by the product owner.

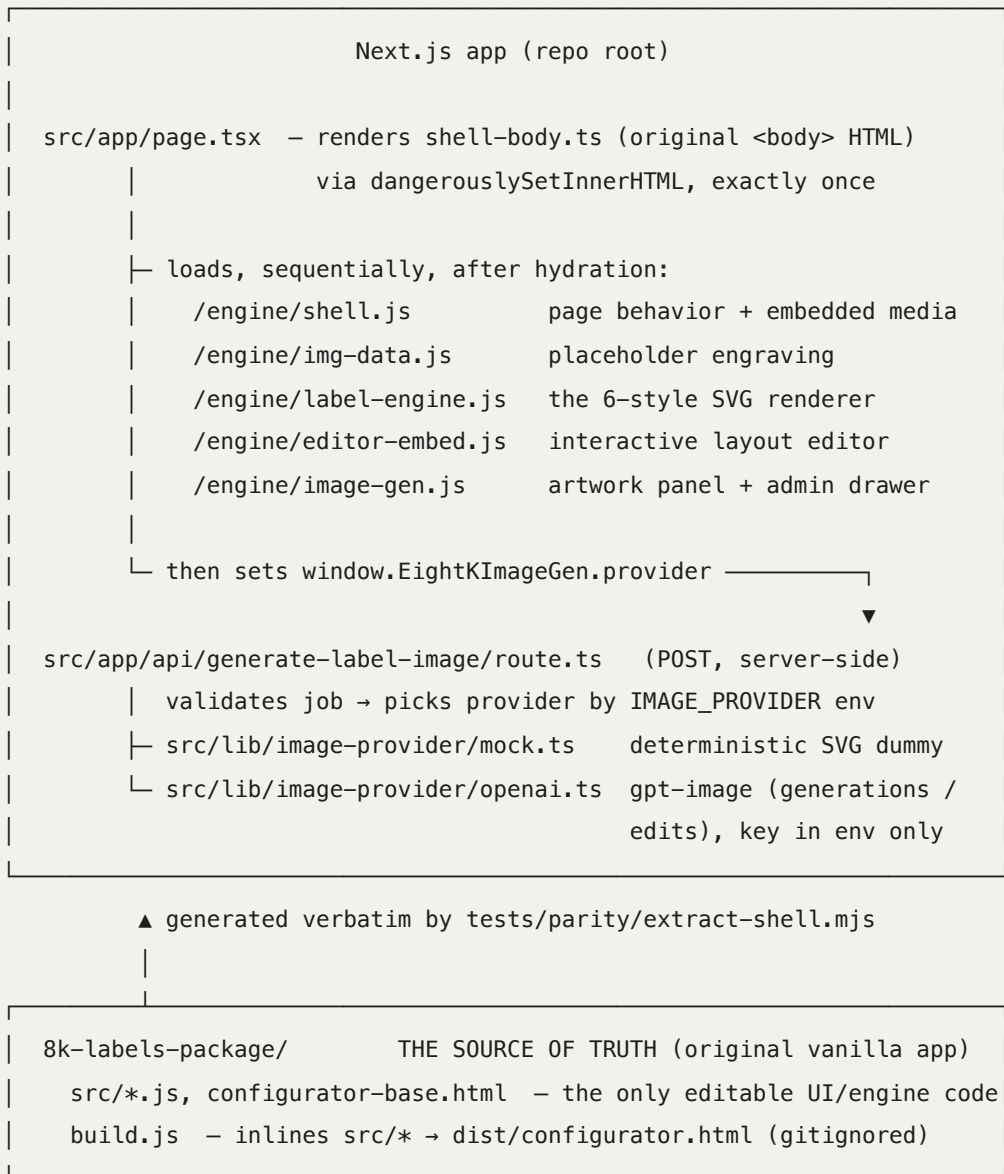
3. Background: why the architecture looks like this

A first migration attempt (repo history up to `85d1b12`) rebuilt the UI as React components by reading the source files, and used a simplified placeholder engine. It **diverged from the real product** — it invented a 5-tab navigation the site never had (the real site has 3 tabs; Gallery and About are toplevel-opened views), rendered labels incorrectly, and re-authored the CSS in Tailwind with visible drift. It was rejected and deleted.

The root cause was treating *source code* as the spec. Source markup contains dead and hidden elements (unreachable panels, `display:none` sections) that only a **rendered page** disambiguates. The restart therefore inverted the approach:

*The built file, rendered in a real browser, is a pixel-level specification. The port **transplants** it byte-for-byte and proves equivalence with automated gates. It never reinterprets.*

4. System architecture



4.1 The host page (`src/app/page.tsx`)

A single client component. It renders the transplanted HTML once and never re-renders it; a `useRef` guard survives React StrictMode's double-invoke in dev. After hydration it appends the five original `<script>`s to the DOM **in the original document order** (they are classic IIFEs, not modules — order matters), then

installs the backend provider through the one hook the original code exposes for exactly this purpose. React owns nothing inside the configurator; the original scripts own the DOM exactly as they do in the single-file version. This eliminates the entire class of "React re-render fights vanilla DOM mutation" bugs.

4.2 Generated artifacts (never hand-edited)

Artifact	Source	Generator
public/engine/*.js (5 files)	<script> blocks of dist, byte-preserved	tests/parity/extract-shell.mjs
src/app/shell-body.ts	<body> HTML of dist minus script/style tags	same
src/app/configurator.css	both <style> blocks of dist, in order	same
src/app/shell-scripts.json	script load order	same

shell-body.ts is emitted as a TypeScript string with every non-ASCII character \u -escaped, because webpack's JSON-module inlining corrupts surrogate pairs (the chat widget's emoji broke the production build when the HTML was imported as .json).

Change workflow: edit 8k-labels-package/src/* → node build.js (in the package) → node tests/parity/extract-shell.mjs → run the parity gates (§7). If the change is intentional, re-baseline the reference artifacts first.

4.3 Layout (src/app/layout.tsx)

Reproduces the original <head> : the Google Fonts link for Hepta Slab (the UI shell font), the page title, and the two global stylesheets. No other framework chrome is added.

5. The label engine (unchanged, hosted verbatim)

public/engine/label-engine.js (1,058 source lines) is a pure function from data to SVG strings; it has no framework dependencies and is the most valuable asset in the codebase, so it is not rewritten — new code calls it through window.LabelEngine .

Key conventions (full detail in 8k-labels-package/CONTINUE-HERE.md §4–5):

- **Units:** 1 unit = 0.1 mm; PT_U ≈ 3.5278 units per PostScript point. viewBox equals the trim size; physical width / height are in mm.
- **Print safety:** 5 mm text margin enforced on all sides; 2 mm bleed drawn beyond the trim; absolute font floor 7 pt with per-element floors above it.
- **Six styles:** Traditional (heritage serif + engraving), Contemporary, Flora & Fauna, Premium (light/dark by seed parity), Minimalist, Artistic/Punk. renderStyleOptions(data, order, {widthMM, heightMM, seed}) returns one option per style; reseeding varies layouts within each style.

- **Determinism:** for a fixed data + size + seed the output SVG is byte-stable — this property is what makes golden testing possible.
- **Fonts:** ~30 Google families loaded via a runtime `@import ( FONTS_URL );` `ensureFonts()` must complete before rendering because the engine sizes text by canvas measurement.

5.1 The font-loading race (operationally important)

`ensureFonts()` injects the `@import` stylesheet and immediately calls `document.fonts.load()`. Until the stylesheet is *parsed*, the families are unregistered — so `load()` resolves as a silent no-op **and** `document.fonts.check()` returns `true` via system-fallback matching. The engine then measures text with fallback metrics and makes different wrap/arc decisions (this produced 34/144 golden failures before diagnosis).

Rule: the only trustworthy readiness signal is actual `FontFace` entries reaching `status === 'loaded'` for each required family, re-invoking `ensureFonts()` each poll. Reference implementation: `src/app/engine-test/page.tsx`.

6. Image generation pipeline

6.1 Client side (original code, untouched)

The winemaker writes a story (`#visionText`) and optionally uploads a sketch (`#sketchFile` → `window.__LABEL_REF__`), then presses **Generate artwork** (`#ig_go`). `EightKImageGen.buildJob()` assembles:

```
job = {
  prompt,                // house art direction + client story + reference note
  negative,               // from the admin Art Direction config
  reference,              // uploaded sketch as data URL, or null
  size: { w, h },
  art: { preset, extra, negative, template },
  data,                   // the wine fields
  vision                  // the raw story text
}
```

The result URL is stored in `window.__LABEL_IMG__` and a `8kRepaint` event re-renders any shown labels with the new artwork in the image slot. A hidden **Art Direction** admin drawer (`?admin=1` or `#art-direction`) controls the prompt template, style preset, house rules and negative prompt.

6.2 Server side (new code)

`POST /api/generate-label-image` validates the job (400 on malformed JSON or missing prompt) and dispatches by the `IMAGE_PROVIDER` environment variable:

- **mock (default, active).** `src/lib/image-provider/mock.ts` returns a deterministic engraving-style SVG data URL derived from an FNV-1a hash of the prompt: hills, sun position and paper tone vary per

prompt; the same vision always produces the same image. It is watermarked "MOCK ARTWORK — PROVIDER NOT WIRED TO A REAL MODEL" so it can never be mistaken for production output. Free and offline — the entire client flow is exercised without model spend.

- `openai` (**written, untested**). `src/lib/image-provider/openai.ts` calls the OpenAI Images API with `OPENAI_API_KEY` from the server environment: `/images/generations` for text-only jobs, `/images/edits` when a reference sketch is present. The negative prompt is folded into the prompt text (the Images API has no negative field); the label aspect ratio maps to the nearest supported size (1536×1024 / 1024×1536 / 1024×1024). Default model `gpt-image-2`, overridable via `OPENAI_IMAGE_MODEL`.

Errors return proper status codes with `{ error }` bodies; the original UI surfaces them through its existing alert path. The key never reaches the client under any configuration.

6.3 Security posture

- API key: server env only (`.env.local` , gitignored; `.env.example` is the committed template). Any key that ever touched a client or a chat is considered burned and must be rotated.
- The admin drawer is currently gated by URL only (`?admin=1`) — acceptable for local development, **must be replaced with real auth before deploy**.
- The generation route has no rate limiting yet — required before any public exposure, since each real call costs money.

7. The parity verification system

The proof system is the heart of the project: it converts "looks right" into checkable facts. All tooling lives in `tests/parity/` and runs on Playwright driving the system Chrome (`channel: 'chrome'` — no browser download).

7.1 Golden SVG corpus (engine equivalence)

`extract-golden.mjs` opens the **original** file and records `renderStyleOptions()` output for a fixed matrix: 2 datasets (full × accented-French fields, minimal) × 3 label sizes (110×80, 80×110, 100×80 mm) × 4 seeds (0, 1, 2, 7) × 6 styles = **144 SVGs**, committed under `tests/parity/golden/`. Corpus determinism was verified by double extraction before first use.

`check-golden.mjs` renders the same matrix through the **ported** engine (via the `/engine-test` harness page) and requires **byte-identical** output on all 144 cases. Current status: **144/144**.

7.2 Screenshot gate (UI equivalence)

`capture-original.mjs` drives either build through ten real UI states — front tab pristine / empty-warning / filled editor / six generated options / lightbox / back tab / bottle tab / admin drawer / gallery / about — and produces full-page screenshots plus a visible-DOM outline. `compare-screens.mjs` pixel-diffs ported vs reference and requires **0.000%** on the nine deterministic states.

The gallery grid is intentionally shuffled (a single `Math.random Fisher– Yates` in `shell.js`), so pixels cannot match run-to-run; the comparator instead asserts the **card set** is identical (captured as sorted text in `gallery-cards.json`). Current status: all states pass.

7.3 End-to-end generation test

`test-imagegen.mjs` exercises the full user journey against a running server: type a vision → click Generate → assert the provider round-trip set `__LABEL_IMG__` and showed the preview → assert mock determinism (same vision = same image, changed vision = different image) → Show Labels → assert the generated artwork is embedded inside the rendered label options.

7.4 What is committed vs regenerable

Committed (the spec)	Gitignored (regenerable output)
<code>tests/parity/reference/</code> (screenshots, option SVGs, DOM outline)	<code>tests/parity/ported/</code>
<code>tests/parity/golden/</code> (144 SVGs + manifest)	<code>tests/parity/diff/</code> , <code>failures/</code>
all <code>tests/parity/*.mjs</code> tooling	<code>8k-labels-package/dist/</code> , <code>.next/</code> , env files

7.5 Command reference

```
npm run dev                # app on :3000
npm run build              # production build
npm run capture:original   # re-baseline reference from the original file
npm run golden:extract     # re-baseline the golden corpus
npm run golden:check       # engine gate (self-starts dev on :3199)
npx next start -p 3200     # then:
npm run capture:ported     #   screenshot the ported app
npm run compare:screens    #   pixel gate
node tests/parity/test-imagegen.mjs http://localhost:3200  # e2e gate
```

8. Design decisions and trade-offs

Decision	Alternative rejected	Rationale
Transplant verbatim; React only hosts	Rewrite UI as React components	The rewrite was tried and diverged; verbatim is provably identical and keeps the engine's byte-stable determinism
Scripts loaded post- hydration via DOM append	<code><script></code> in SSR head/body	Scripts mutate the DOM; running them before hydration causes React mismatches or wiped mutations
Body HTML as <code>\u</code> - escaped <code>.ts</code> string	<code>.json</code> import	webpack corrupts surrogate pairs (emoji) when inlining JSON modules — broke <code>next build</code>
Original <code>@import</code> font mechanism kept	<code>next/font</code>	The engine sizes text by measurement; <code>next/font</code> changes loading/metrics timing and risks wrap drift
Golden SVGs byte- compared	Visual/approximate compare	Byte equality is the strongest possible claim and free to check; determinism makes it feasible
Gallery compared by card set	Seeding/patching the shuffle	Patching would modify verbatim code; the set check verifies content without touching it
Mock provider returns SVG data URL server-side	Client-side placeholder (original behavior)	Exercises the real network path, API validation, and error handling end-to-end at zero cost
Commit reference + golden (~20 MB)	Regenerate per machine	Cross-machine font/browser variance would silently move the baseline; committed spec pins it

9. Roadmap

1. **Real generation hardening** — first live run of the OpenAI path (verify the model id), plus rate limiting / abuse protection on the route.
2. **Art Direction persistence** — server-side storage for the admin config (currently in-memory, resets on reload), loaded on page init.
3. **Admin auth** — replace `?admin=1` with a real authenticated route.
4. **Deferred product items** — logo placement rules, per-style image treatments, auto-generate on Show Labels (cost/latency decision), mobile verification, extending the package's own `verify.js` sweep to all 6 styles.

10. Glossary

Term	Meaning
Transplant	Byte-preserving extraction of the built page into Next.js-hosted files
Golden corpus	The 144 committed SVGs the ported engine must reproduce exactly
Parity gate	Any automated check that must pass before a change lands
Provider	Server-side implementation of image generation behind the API route
Job	The <code>buildJob()</code> payload: prompt + reference + wine data + vision
Re-baseline	Re-capturing reference/golden after an <i>intentional</i> source change